SOFTWARE REQUIREMENTS SPECIFICATION

Toxic Comment Classification System Using Deep Learning

Developer-Oriented Requirements & System Design

A concise implementation-focused specification covering system behavior, ML pipeline, architecture, interfaces, testing, and deployment.

Name	M MOHAN
Roll No.	R220325
Email	r220325@rguktrkv.ac.in
Department	Computer Science
College/Institution	Rajiv Gandhi University of Knowledge Technologies
Semester	E-3 Sem-1
Faculty	Lingamurthy Sir

1. ABSTRACT

The Toxic Comment Classification System is a Deep Learning-based NLP application that automatically analyzes user-generated text and predicts whether the comment contains one or more forms of toxic behavior.

The system accepts a text comment, validates and preprocesses it, converts the text into a representation suitable for the selected model, performs inference, and returns toxicity categories with probability/confidence information where applicable.

The project treats toxicity detection as a multi-label classification problem because a single comment can belong to multiple categories. The supported categories are toxic, severe toxic, obscene, threat, insult, and identity hate. The intended use is moderation support rather than fully autonomous moderation.

2. PROBLEM STATEMENT

Online platforms receive large volumes of user-generated comments. Manual review is difficult to scale and can be slow or inconsistent. Keyword and rule-based approaches can also miss context, spelling variations, and complex language.

Requirement: Build a system that automatically detects and classifies toxic comments using Natural Language Processing and Deep Learning techniques.

3. OBJECTIVES

- Automate toxic-comment classification.
- Prepare and preprocess a labeled text dataset.
- Convert comments into numerical model input.
- Train and evaluate a Deep Learning classifier.
- Support multi-label toxicity prediction.
- Return category-level probabilities/confidence scores.
- Expose the prediction capability through a usable interface/API.
- Provide a foundation for integration with online moderation systems.

4. SCOPE

4.1 In Scope

- Text-based comment classification
- Dataset preparation and exploratory analysis
- NLP preprocessing and tokenization
- Text/vector representation
- Deep Learning model development
- Training, validation, and testing
- Multi-label prediction
- Model evaluation
- Prediction API/UI
- Result display

4.2 Out of Scope

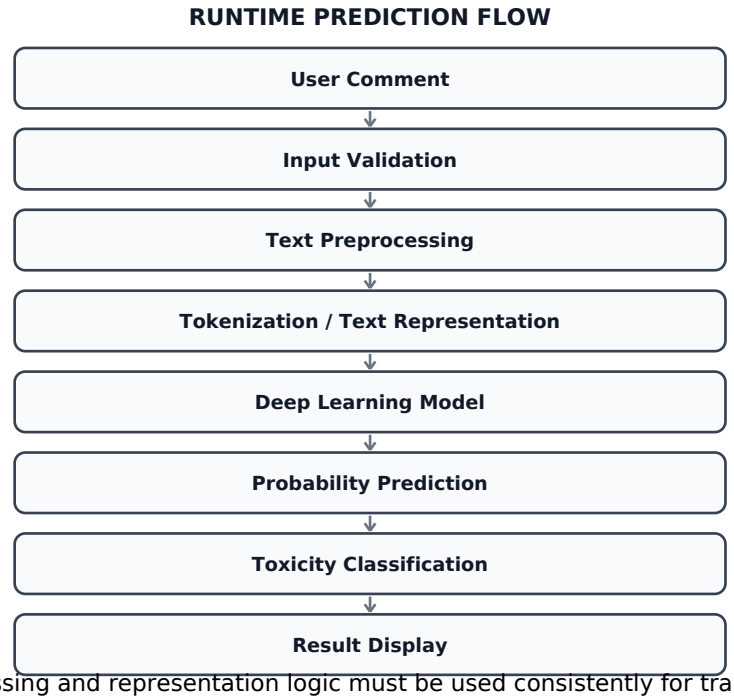
- Image, video, and audio toxicity detection
- Automatic account banning
- Legal determination of harmful content
- Human-level contextual understanding
- Fully autonomous moderation without human review

5. EXISTING VS PROPOSED APPROACH

Area	Existing Approach	Proposed Approach
Moderation	Manual review	Automated model-assisted classification
Filtering	Keyword / rules	NLP + Deep Learning
Context	Limited	Learned from training data
Scale	Difficult for large volumes	Designed for automated inference
Output	Moderator decision	Toxicity labels + confidence

6. SYSTEM FUNCTIONAL FLOW

The runtime path below is the primary developer view of the application.



The same preprocessing and representation logic must be used consistently for training data and prediction input.

7. TOXICITY LABELS

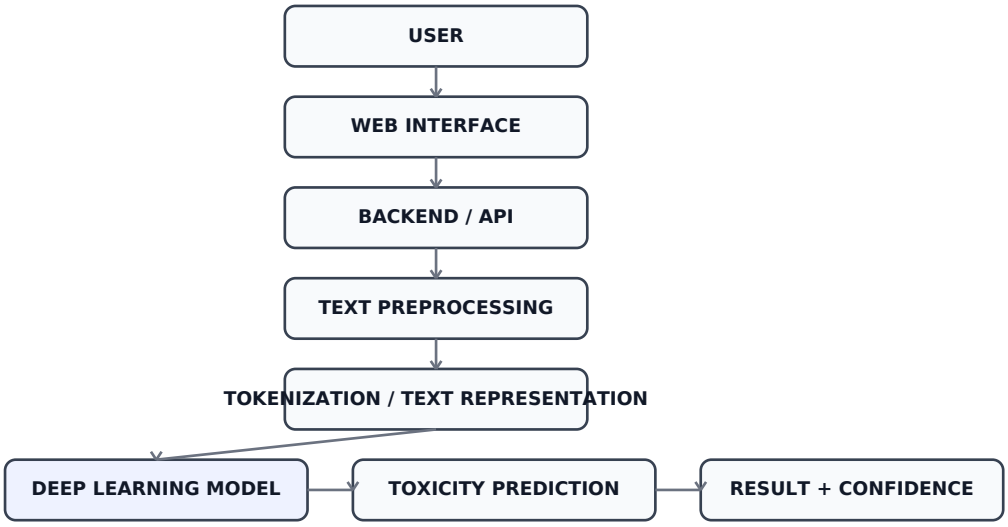
Label	Meaning
Toxic	General toxic or abusive language
Severe Toxic	Highly severe toxic content
Obscene	Obscene or vulgar content
Threat	Threatening content
Insult	Insulting content
Identity Hate	Hate targeting an identity or group

8. USER / MODERATOR USE CASES

- User – Submit Comment: enter a comment and submit it for classification.
- User – View Classification: view toxicity status, predicted categories, and confidence scores.
- Moderator – Review Flagged Comment: review potentially toxic content and take the appropriate moderation action.
- Administrator – Update Model: manage data, train/evaluate the model, and deploy an acceptable model version.

9. SYSTEM ARCHITECTURE

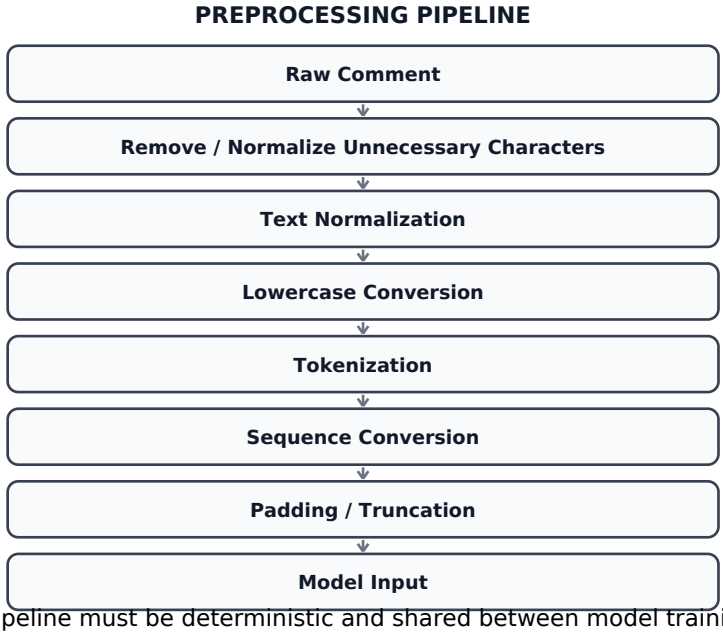
The architecture separates user interaction, API processing, NLP preprocessing, model inference, and result presentation.



9.1 Main Components

- Web Interface: accepts user comments and displays results.
- Backend / API: receives requests and invokes the prediction pipeline.
- Preprocessing: normalizes and prepares input text.
- Tokenizer / Representation: converts text into model-compatible numerical input.
- Deep Learning Model: performs multi-label classification.
- Prediction Layer: converts model output into toxicity categories and confidence information.

10. DATA PREPROCESSING



The preprocessing pipeline must be deterministic and shared between model training and inference.

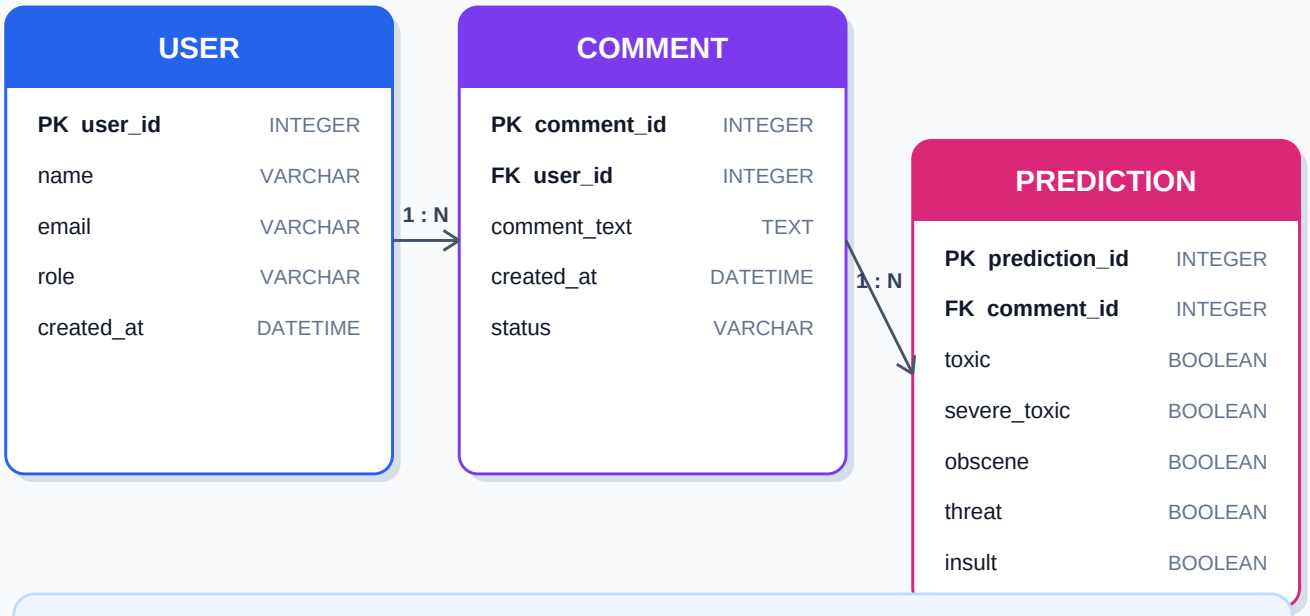
11. TEXT REPRESENTATION

- TF-IDF: numerical representation based on term importance.
- Word Embeddings: dense vector representation such as Word2Vec, GloVe, or FastText.
- Neural Embeddings: representations learned as part of model training.
- Transformer Representations: contextual representations such as BERT.

DATABASE DESIGN — ENTITY RELATIONSHIP DIAGRAM

PROPOSED DATA MODEL

The proposed data model supports user comments, classification predictions, and category-level toxicity results. A user can submit multiple comments, and each comment can have one or more associated prediction records.



RELATIONSHIPS

USER 1:N COMMENT — one user may submit multiple comments.
COMMENT 1:N PREDICTION — a comment may have multiple prediction records over time.

IMPLEMENTATION NOTES

- Primary keys uniquely identify users, comments, and prediction records.
- Foreign keys maintain referential integrity between related entities.
- Prediction stores the six supported toxicity labels as category-level outputs.
- Confidence information can be stored as JSON or a normalized child table.
- The model itself is deployed as a service; model files are not stored in the relational schema.

DESIGN STATUS

Proposed logical database design derived from the SRS entities, prediction outputs, and module responsibilities.

12. MODEL REQUIREMENTS

The system shall use a trained Deep Learning model for toxicity classification.

Candidate	Role in the project
ANN	Baseline neural classifier
CNN	Alternative text classification architecture
LSTM / GRU	Sequence-oriented text classification
Bi-LSTM	Bidirectional sequence modeling
BERT / Transformer	Advanced contextual representation and classification

The source specification identifies LSTM/Bi-LSTM as an initial baseline option and Transformer-based models as an advanced alternative. The final architecture should be selected based on project evaluation results.

13. FUNCTIONAL REQUIREMENTS

ID	Requirement	Developer Interpretation
FR-01	User Input	Accept a text comment.
FR-02	Input Validation	Reject or handle empty and excessively long input.
FR-03	Preprocessing	Run the configured NLP preprocessing pipeline.
FR-04	Tokenization	Convert processed text using the selected tokenizer.
FR-05	Classification	Run inference using the trained model.
FR-06	Multi-Label Prediction	Allow multiple toxicity labels for one comment.
FR-07	Confidence Score	Return probability/confidence values for categories.
FR-08	Result Display	Present classification results clearly.
FR-09	Error Handling	Return an appropriate error when prediction fails.
FR-10	Prediction Logging	Optionally store prediction information for monitoring/analysis.

14. NON-FUNCTIONAL REQUIREMENTS

Requirement	Expected Behavior
Performance	Prediction should return within an acceptable response time under normal load.
Accuracy	Model performance should be measured using defined evaluation metrics.
Reliability	Same input/model configuration should produce consistent results.
Scalability	Architecture should allow additional prediction requests/users in future.
Security	Protect inputs, APIs, administrative functions, stored data, and model resources.
Usability	Interface should be simple for users without ML knowledge.
Maintainability	Dataset, preprocessing, tokenizer, model, backend, and frontend should remain modular.

15. INPUT AND OUTPUT CONTRACT

15.1 Input

Primary input: a text comment. The request is validated before model inference and then passed through the configured preprocessing pipeline.

15.2 Output

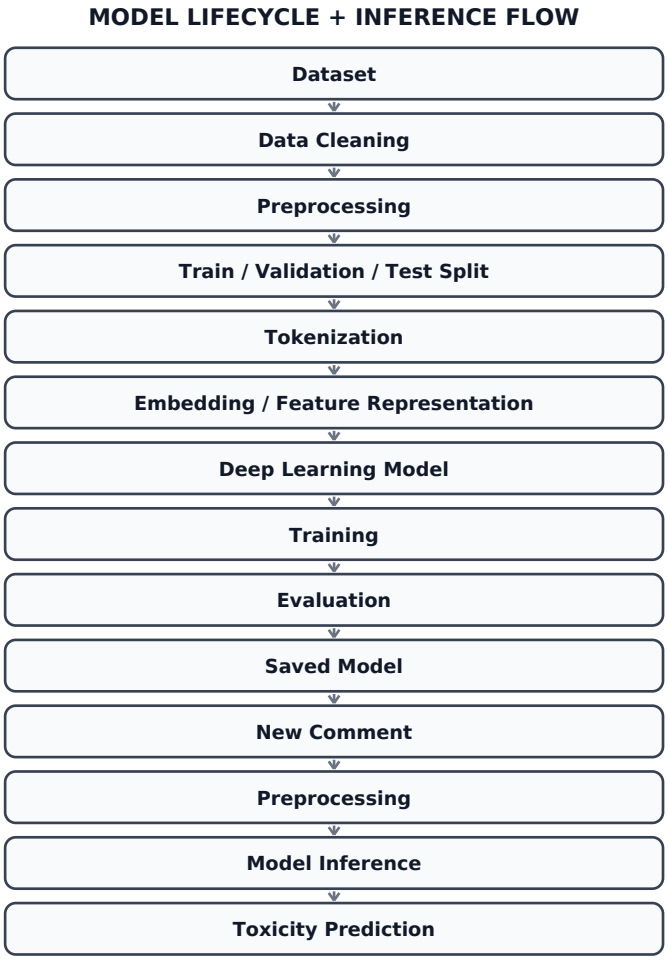
The system returns the predicted toxicity categories and, where applicable, probability/confidence scores.

Category	Example Result
Toxic	YES
Severe Toxic	NO
Obscene	NO
Threat	NO
Insult	YES
Identity Hate	NO

16. MODULE RESPONSIBILITIES

Module	Responsibility
Data Collection	Obtain the labeled dataset.
Data Analysis	Inspect size, class distribution, missing values, duplicates, and label distribution.
Preprocessing	Clean and prepare text.
Text Representation	Convert text into numerical model input.
Model Training	Train the selected Deep Learning model.
Model Evaluation	Measure classification performance.
Prediction	Run inference on new comments.
UI / API	Accept input and return/display predictions.
Monitoring / Logging	Optionally record prediction information and monitor behavior.

17. TRAINING AND INFERENCE DATA FLOW



18. MODEL EVALUATION

Because toxicity datasets may be class-imbalanced, model quality should not be judged using accuracy alone.

- Accuracy: percentage of correct predictions.
- Precision: proportion of predicted positives that are actually positive.
- Recall: proportion of actual positives that are detected.
- F1-score: harmonic balance between precision and recall.
- ROC-AUC: optional measure of discrimination performance.
- Confusion Matrix: useful for analyzing classification errors.

19. DATASET REQUIREMENTS

The system requires a labeled dataset containing comments and corresponding toxicity labels. The source specification identifies the Jigsaw Toxic Comment Classification dataset as a suitable dataset.

- Labeled comments
- Multiple toxicity labels
- Representative training examples
- Consistent preprocessing between training and inference

20. TESTING STRATEGY

Test Level	What to Verify
Unit Testing	Preprocessing, tokenization, prediction functions, and input validation.
Integration Testing	Frontend, API, preprocessing, and model interaction.
Model Testing	Performance on unseen test data.
System Testing	Complete end-to-end application behavior.
Performance Testing	Latency, throughput, resource usage, and multiple requests.

21. ERROR HANDLING

Condition	Expected Response
Empty comment	Display validation message
Excessively long comment	Display input-length error
Invalid request	Return appropriate error
Model unavailable	Display service-unavailable message
Prediction failure	Display prediction error
Server failure	Return appropriate server response

22. SECURITY AND PRIVACY

22.1 Security

- Validate user input.
- Protect APIs from unauthorized access.
- Authenticate administrative functions.
- Protect stored prediction data.
- Avoid exposing internal model files.
- Log system errors securely.

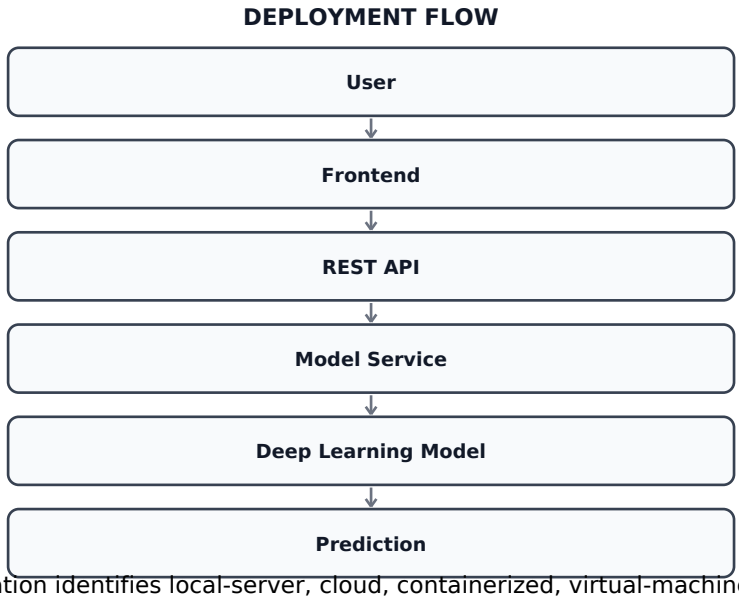
22.2 Privacy

- Store only required information.
- Restrict access to stored comments.
- Avoid unnecessary personal information.
- Apply appropriate data-retention policies.
- Protect stored data from unauthorized access.

23. ACCEPTANCE CRITERIA

- User can submit a text comment.
- Input validation works correctly.
- Text is preprocessed using the configured pipeline.
- The trained model performs inference successfully.
- Defined toxicity categories are supported.
- Multiple labels can be returned for one comment.
- Results are displayed correctly.
- Confidence scores are generated where applicable.
- Model performance is evaluated on unseen test data.
- System errors are handled appropriately.
- Application executes successfully in the target environment.

24. DEPLOYMENT ARCHITECTURE

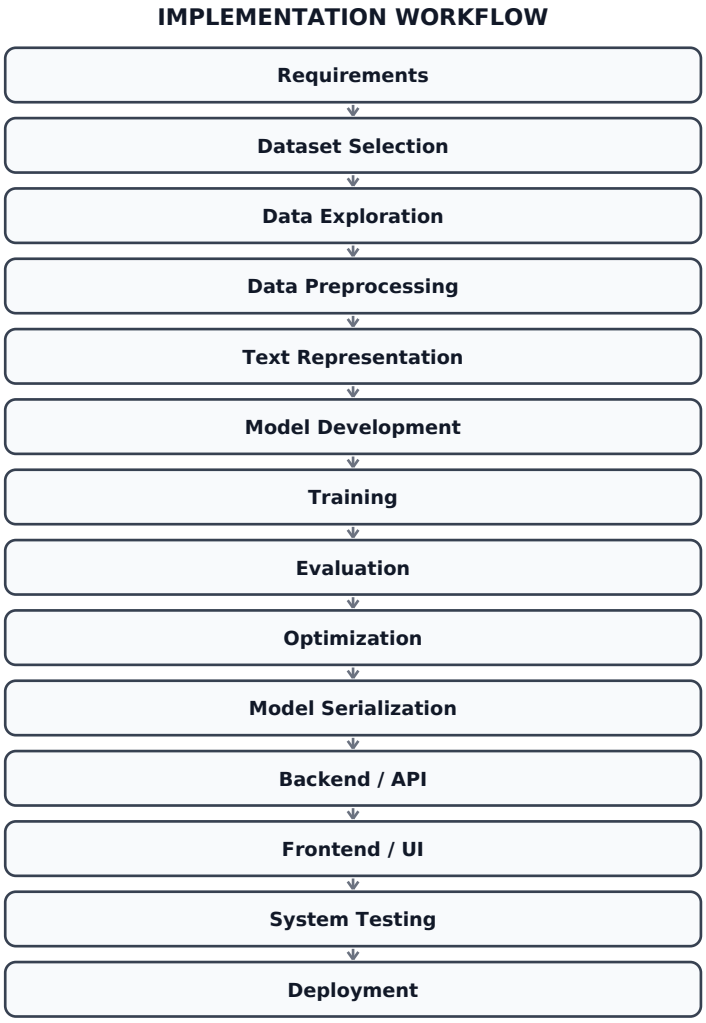


The source specification identifies local-server, cloud, containerized, virtual-machine, and Kubernetes-based deployment as possible environments.

25. TECHNOLOGY STACK

Layer	Technologies Identified in the SRS
Programming	Python 3.x
Data Processing	NumPy, Pandas
Machine Learning	Scikit-learn
Deep Learning	TensorFlow / Keras or PyTorch
NLP	NLTK, spaCy, Hugging Face Transformers if required
Visualization	Matplotlib
Backend	Flask or FastAPI
Frontend	HTML, CSS, JavaScript, or React
Database (Optional)	MySQL, PostgreSQL, or MongoDB
Deployment	Cloud / Docker / Linux

26. DEVELOPMENT WORKFLOW



27. CONCLUSION

The proposed system is a modular Deep Learning application for automated toxic-comment classification. Its core implementation consists of a text-processing pipeline, a trained multi-label classifier, an inference layer, and a user/API interface.

The document is intentionally focused on the information a developer needs to understand the system: what enters the system, how it is transformed, which modules are responsible for each stage, what the model returns, how the system is tested, and how it can be deployed.